

Rapport du Projet Base de données Avancées

Participants: Martrenchar Luc, Doyhenart Xan, Abouleila Selim, Nguema André

1. Introduction

BoardTrust est une plateforme complète de location de jeux de société, développée en full-stack avec un back-end en Node.js/Express et une base de données MySQL, ainsi qu'un front-end en JavaScript, HTML et CSS. Elle permet aux utilisateurs de parcourir un catalogue de jeux fourni par un dataset de l'Efrei, de les louer selon le stock disponible, de laisser des commentaires, et de consulter leur historique de location. L'application inclut également un système d'authentification avec création de compte, connexion et déconnexion, et propose une interface responsive adaptée aux mobiles.

2. Conception de la base de données

2.1 Modèle conceptuel et logique (ERD)

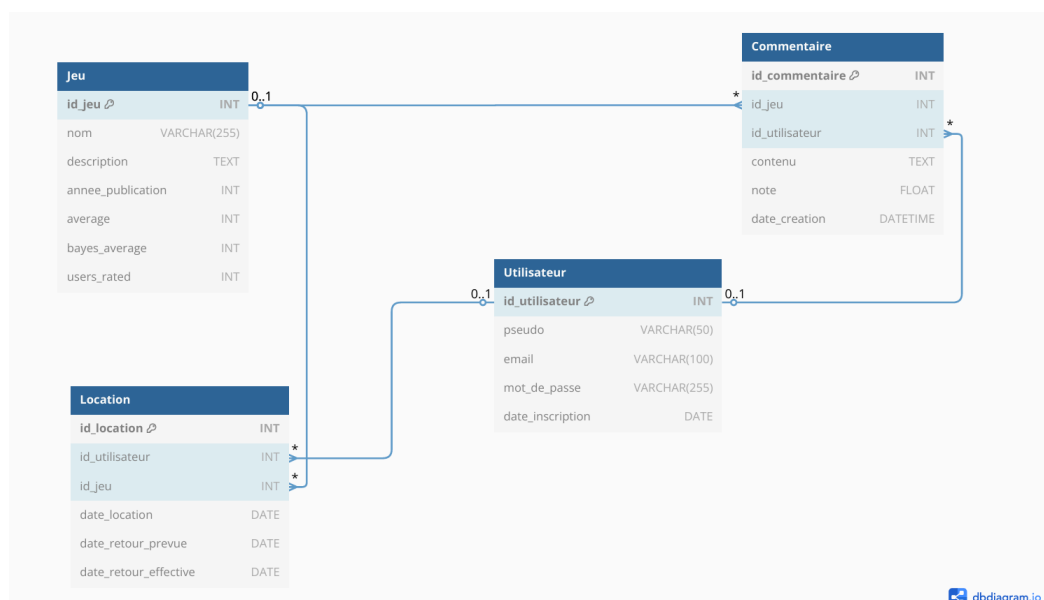


Table Jeu

- id_jeu : entier, clé primaire, auto-incrémentée
- nom : chaîne de caractères (255)
- description : texte
- annee_publication : entier
- average : entier
- bayes_average : entier
- users Rated : entier

Table Utilisateur

- id_utilisateur : entier, clé primaire, auto-incrémentée
- pseudo : chaîne de caractères (50), unique
- email : chaîne de caractères (100), unique
- mot_de_passe : chaîne de caractères (255)
- date_inscription : date

Table Commentaire

- id_commentaire : entier, clé primaire, auto-incrémentée
- id_jeu : entier, clé étrangère vers **Jeu(id_jeu)**
- id_utilisateur : entier, clé étrangère vers **Utilisateur(id_utilisateur)**
- contenu : texte
- note : flottant
- date_creation : date et heure

Table Location

- id_location : entier, clé primaire, auto-incrémentée
- id_utilisateur : entier, clé étrangère vers Utilisateur(id_utilisateur)
- id_jeu : entier, clé étrangère vers Jeu(id_jeu)
- date_location : date
- date_retour_prevue : date
- date_retour_effective : date

2.2 Exemple de procédure stockée

```
DELIMITER //
```



```
> CREATE PROCEDURE LouerJeu (  
    IN p_id_utilisateur    INT,  
    IN p_id_jeu            INT,  
    IN p_date_retour_prevue DATE  
- )  
> BEGIN  
    DECLARE v_stock INT;  
  
    -- get current stock  
    SELECT stock  
        INTO v_stock  
        FROM Jeu  
        WHERE id_jeu = p_id_jeu  
        FOR UPDATE; -- lock the row to avoid race conditions
```

```

-- refuse if none left
IF v_stock <= 0 THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Jeu en rupture de stock';
ELSE
    -- otherwise insert & decrement as before
    INSERT INTO Location (
        id_utilisateur,
        id_jeu,
        date_location,
        date_retour_prevue,
        date_retour_effective
    ) VALUES (
        p_id_utilisateur,
        p_id_jeu,
        CURDATE(),
        p_date_retour_prevue,
        NULL
    );

    UPDATE Jeu
        SET stock = stock - 1
        WHERE id_jeu = p_id_jeu;
END IF;
END //
DELIMITER ;

```

Il s'agit de la procédure qui permet de louer un jeu. Elle prend en paramètres l'identifiant de l'utilisateur, l'identifiant du jeu et la date de retour prévue.

2.3 Exemple de trigger

```
DELIMITER //
```



```
CREATE TRIGGER after_location_insert  
AFTER INSERT ON Location  
FOR EACH ROW  
BEGIN  
    INSERT INTO HistoriqueLocation (  
        id_utilisateur,  
        id_jeu,  
        date_location,  
        date_retour_prevue,  
        date_retour_effective,  
        action  
    ) VALUES (  
        NEW.id_utilisateur,  
        NEW.id_jeu,  
        NEW.date_location,  
        NEW.date_retour_prevue,  
        NULL,  
        'Location créée'  
    );  
END //
```



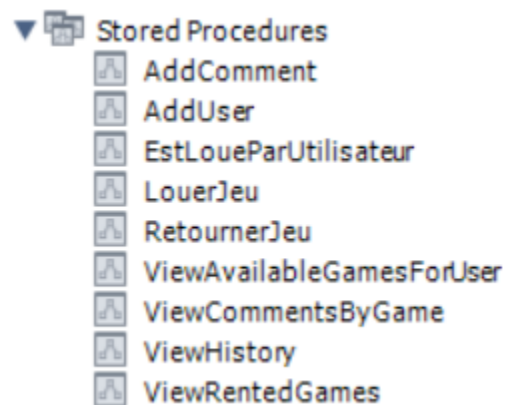
```
DELIMITER ;
```

Il s'agit du trigger qui met à jour l'historique de location lorsqu'un jeu est loué.

2.4 Tous les procédures / triggers

Procedures:

- LouerJeu (id_jeu, id_location, date_retour_prevue)
- RetournerJeu (id_jeu, id_utilisateur)
- ViewAvailableGamesForUser (id_utilisateur)
- AddComment (id_utilisateur, id_jeu)
- ViewCommentsByGame (id_jeu)
- ViewRentedGames (id_utilisateur, id_jeu)
- ViewHistory (id_utilisateur)



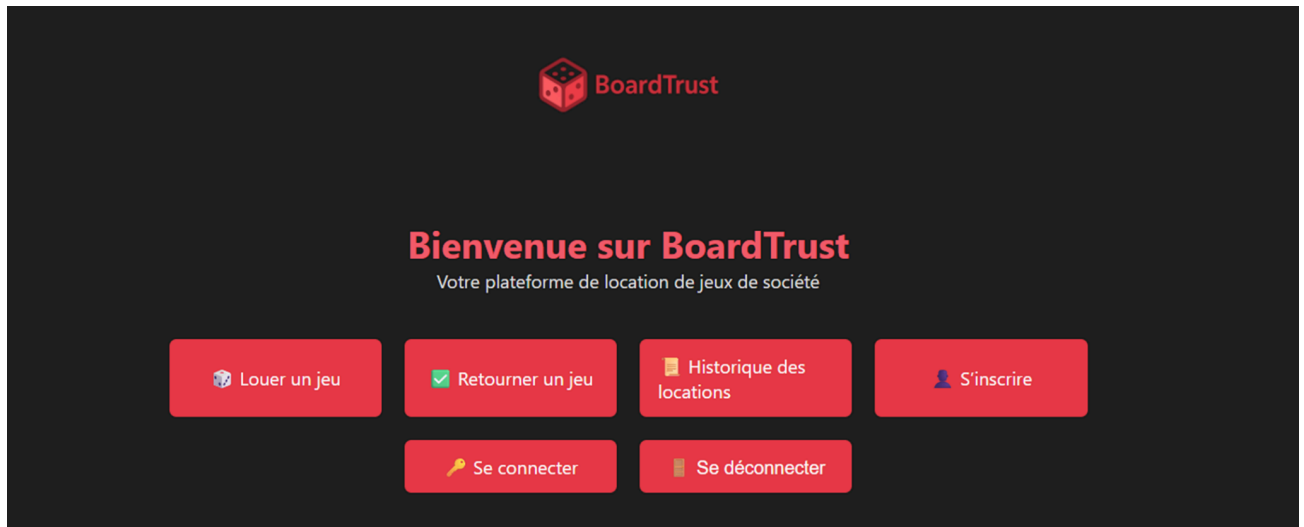
Triggers:

- After_location_Insert: Met à jour la table historique après ajout sur la table des locations
- After_game_return: Met à jour la table historique après retrait sur la table des location

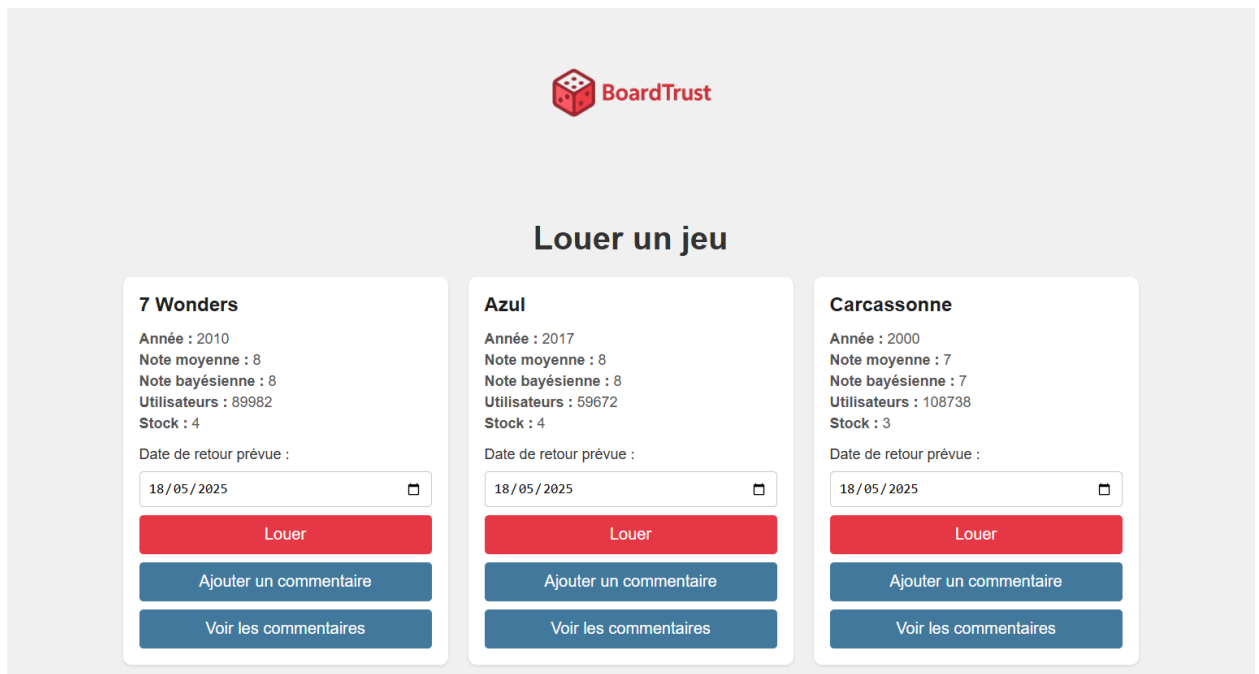
3. Le site web

3.1 Pages principales (avec captures d'écran)

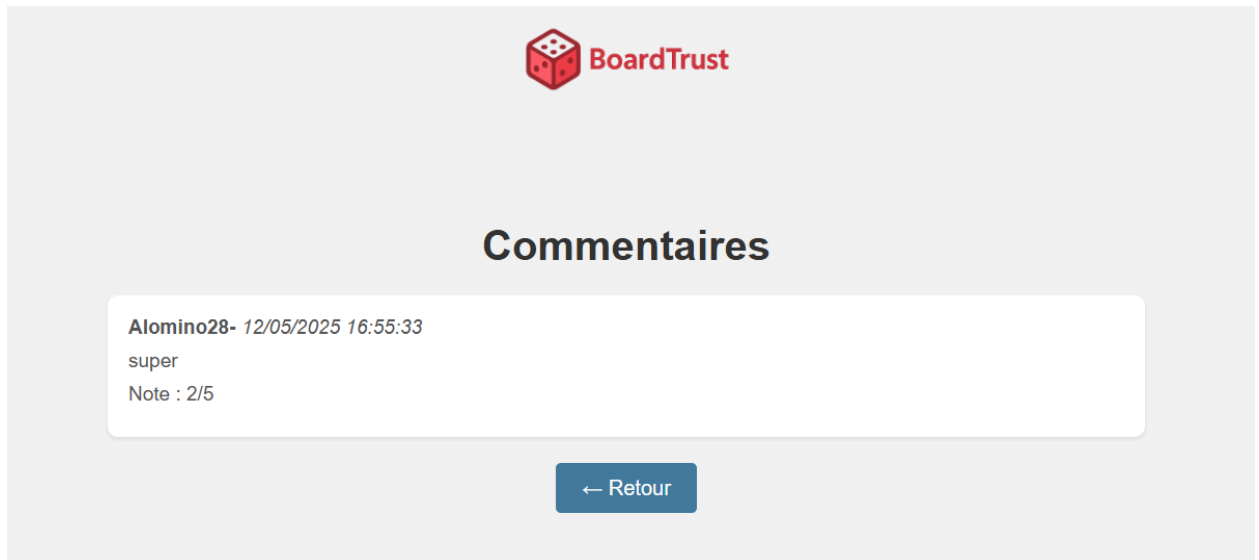
- Accueil (main.html)



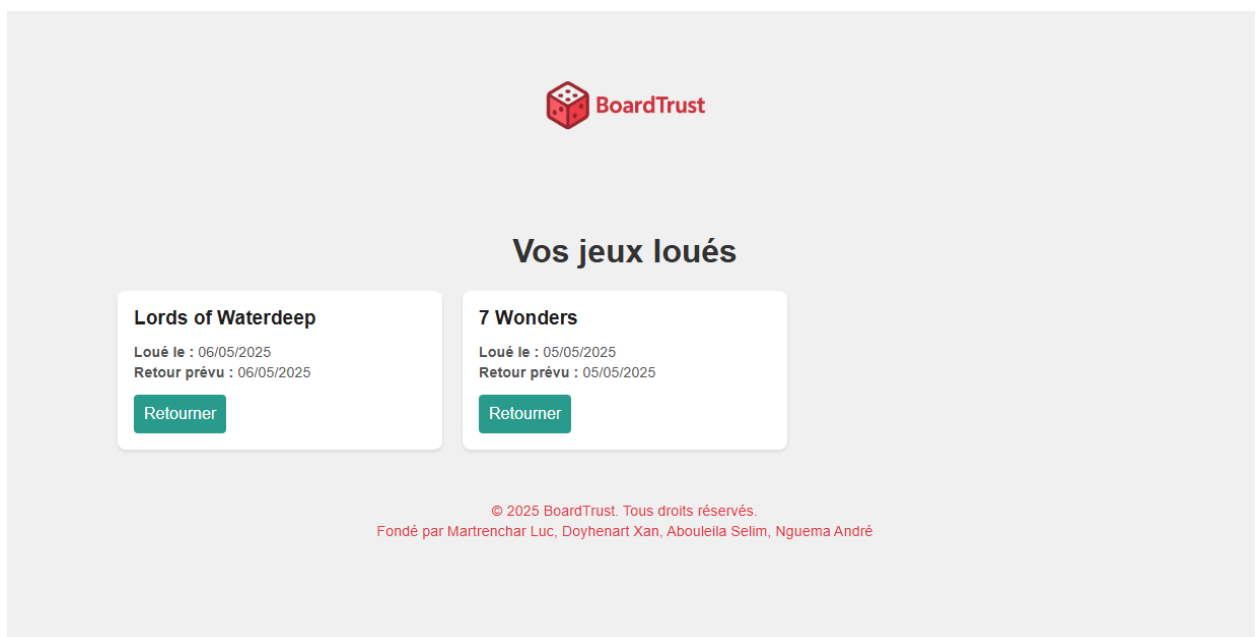
- Page de location (rent.html)



- Page de consultation des commentaires (comments.html)



- Page pour retourner un jeu loué (retour.html)



- Page pour s'inscrire (retour.html)
- Page pour se connecter (login.html)

- Page pour voir l'historique de location (history.html)



Historique des locations

Game name	Date location	Date retour prevue	Date retour effective
Azul	12/05/2025	12/05/2025	12/05/2025
Agricola	12/05/2025	12/05/2025	12/05/2025
Azul	12/05/2025	12/05/2025	12/05/2025
Agricola	12/05/2025	15/05/2025	12/05/2025
Lords of Waterdeep	06/05/2025	06/05/2025	N/A
7 Wonders	05/05/2025	05/05/2025	N/A
Catan	05/05/2025	05/05/2025	12/05/2025
Carcassonne	05/05/2025	09/05/2025	05/05/2025
Catan	05/05/2025	05/05/2025	05/05/2025

3.2 Fonctionnalités clés du site

- Connexion/inscription utilisateur
- Louer un jeu (stock vérifié)
- Retourner un jeu (stock incrément)
- Commenter un jeu
- Visualiser les commentaires
- Affichage du stock
- Gestion des jeux déjà loués

4. Difficultés rencontrées

Nous avons rencontré plusieurs difficultés lors de la réalisation de ce projet. Parmi celles-ci, on peut citer :

- La connexion du site à la base de données, ce qui n'a pas été facile car cela nécessitait un protocole précis.
- La simplification en profondeur de la structure de la base de données entre la version alpha et la version finale. Nous nous sommes rendu compte que trop de tables rendaient le projet difficile à réaliser dans le temps imparti.
- La création d'un design CSS agréable pour améliorer l'interface du site.
- La réalisation tardive que la procédure d'authentification fonctionnait uniquement en HTTP, ce qui nous a obligés à tout reprendre et à l'adapter pour HTTPS, le protocole sécurisé de transfert de données (HyperText Transfer Protocol Secure).
- Le développement du back-end et la gestion de nombreux bugs, notamment lors de la mise en place de l'authentification (choix des bons modules à importer), ou encore le bon usage des requêtes POST et GET.
- L'hébergement en ligne du site. Même si ce n'était pas obligatoire, nous avons décidé de le faire pour donner un aspect plus réaliste au projet et permettre un accès public, tout en le mettant en valeur sur GitHub.

5. Conclusion

Le projet **BoardTrust** nous a permis de mettre en pratique l'ensemble des compétences acquises en base de données et en développement web full-stack. De la conception du schéma relationnel à l'implémentation des procédures stockées, en passant par la création d'un site web fonctionnel et interactif, nous avons pu aborder toutes les étapes clés d'un projet complet. Malgré les difficultés techniques rencontrées, ce projet nous a appris à collaborer efficacement, à résoudre des problèmes concrets et à produire une application utile et accessible. Nous sommes fiers du résultat obtenu et de la possibilité d'avoir pu proposer une démonstration en ligne ouverte au public.